

Best Practice for Workflows

www.therefore.net
© 2024 Therefore Corporation

All rights reserved. No parts of this work may be reproduced in any form or by any means - graphic, electronic, or mechanical, including photocopying, recording, taping, or information storage and retrieval systems - without the written permission of the publisher.

Windows® is a registered trademark of Microsoft Corporation in the United States and/or other countries.

Any other 3rd Party products that are referred to in this document may be either trademarks and/or registered trademarks of the respective owners. The publisher and the author make no claim to these trademarks.

While care has been taken in the preparation of this document, the publisher and the author assume no responsibility for errors or omissions, or for damages resulting from the use of information contained in this document.

VERSION: 2024 - 01

Table of Contents

1. Introduction	4
2. Patterns	5
2.1 Pattern 1 - Correctly using Actions (less is more)	5
2.1.1 Example 1	6
2.1.2 Example 2	8
2.2 Pattern 2 - Wait for an index data change / flag	10
2.2.1 Example	11
2.3 Pattern 3 - Handling automatic task overdues	12
2.3.1 Example	13
2.4 Pattern 4 - Handling of manual task overdues	15
2.4.1 Example	16
2.5 Pattern 5 - Execute script in a loop	19
2.5.1 Example	20
2.6 Pattern 6 - Execute script in a loop with advanced timing	21
2.6.1 Example	22
2.7 Pattern 7 - Do not start a workflow for certain documents	24
2.7.1 Example	24
2.8 Pattern 8 - Several users implementing the same task in parallel	24
2.8.1 Example	25
2.9 Pattern 9 - Synchronizing Sub-workflows	27
2.9.1 Example	28
2.10 Pattern 10 - Voting	30
2.10.1 Example 1	31
2.10.2 Example 2 (Advanced)	32
2.11 Pattern 11 - Abort a long lasting action	34
2.11.1 Example	35

1. Introduction

The purpose of this document is to demonstrate best practices, patterns and formulas for implementing workflows in Therefore™ in order to achieve the most effective outcome under various circumstances.

2. Patterns

2.1 Pattern 1 - Correctly using Actions (less is more)

The user can select actions shown in the UI (as per the samples below), where the user can take a decision or deliberate about the subsequent route of the workflow process. It is best practice to limit the number of available actions to the ones where users have to take decision and hide the actions, which are validated/set by business rules only. As an example in an approval step the user can decide to approve or reject the invoice. If an invoice needs a second approval through management, it is decided by business rules only and not by the user – so this option would not need to be shown in the UI.

As per the case in [Example 1](#):

An improper set of options in the UI may include:

- Approve
- Approve and send to management (for Invoices > 1000€)
- Reject

A more appropriate set of options in the UI may include:

- Approve
- Reject

As per the case in [Example 2](#), there may be multiple routes out of a single task, but also by defined business rules that ensure that only **ONE** choice in the UI is selectable. Hence there would be no value in providing all selections to the user:

An improper example of options in the UI may include:

- Route 1
- Route 2
- Route 3
- Route 4

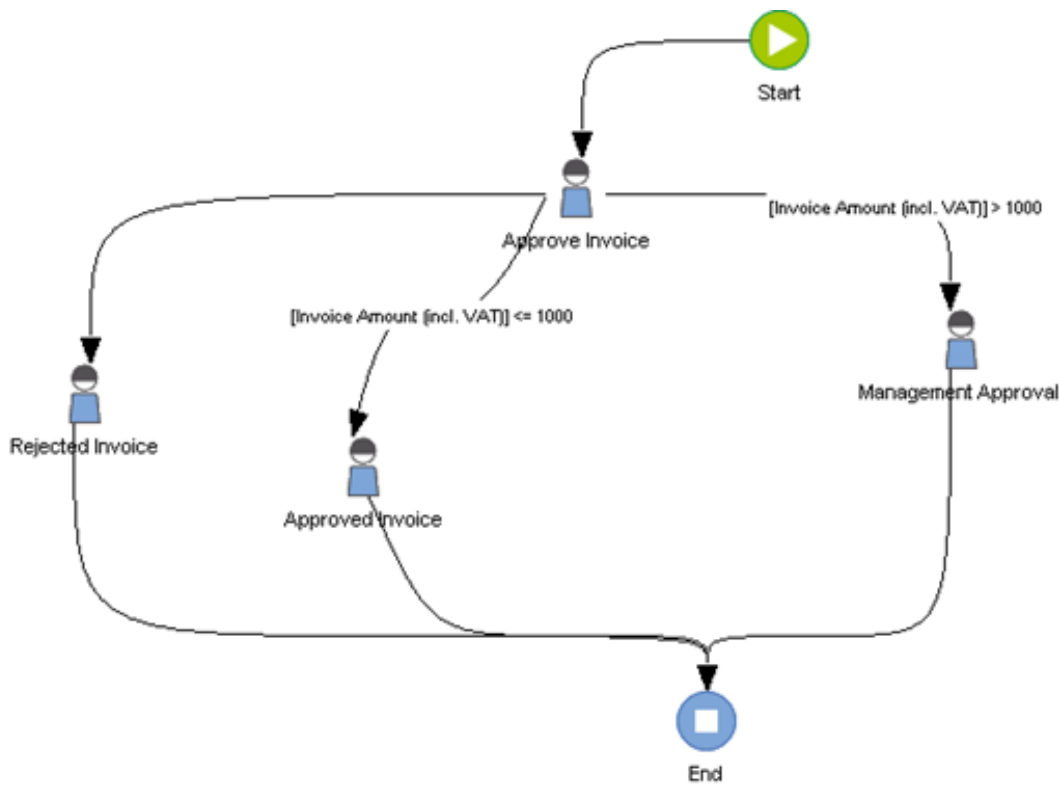
A more appropriate option in the UI may include:

- Continue workflow

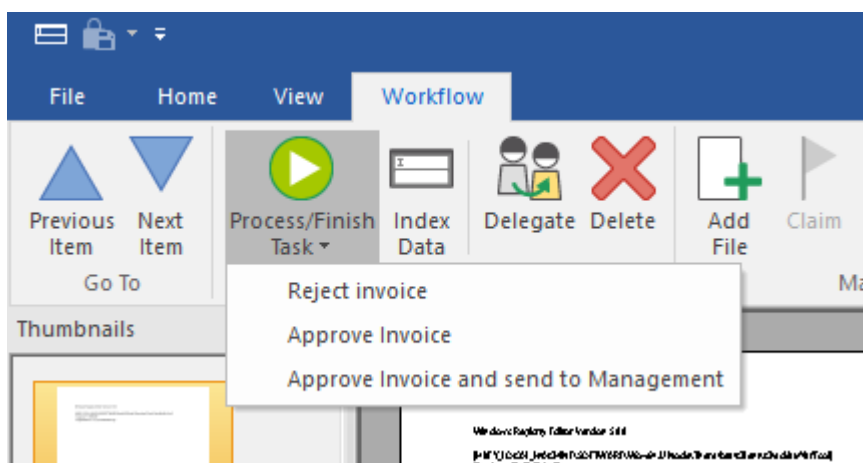
This can easily be achieved by using a pattern where the Manual Task is combined with a Routing Task.

2.1.1 Example 1

A common way in which such workflows are implemented is demonstrated by the below example:

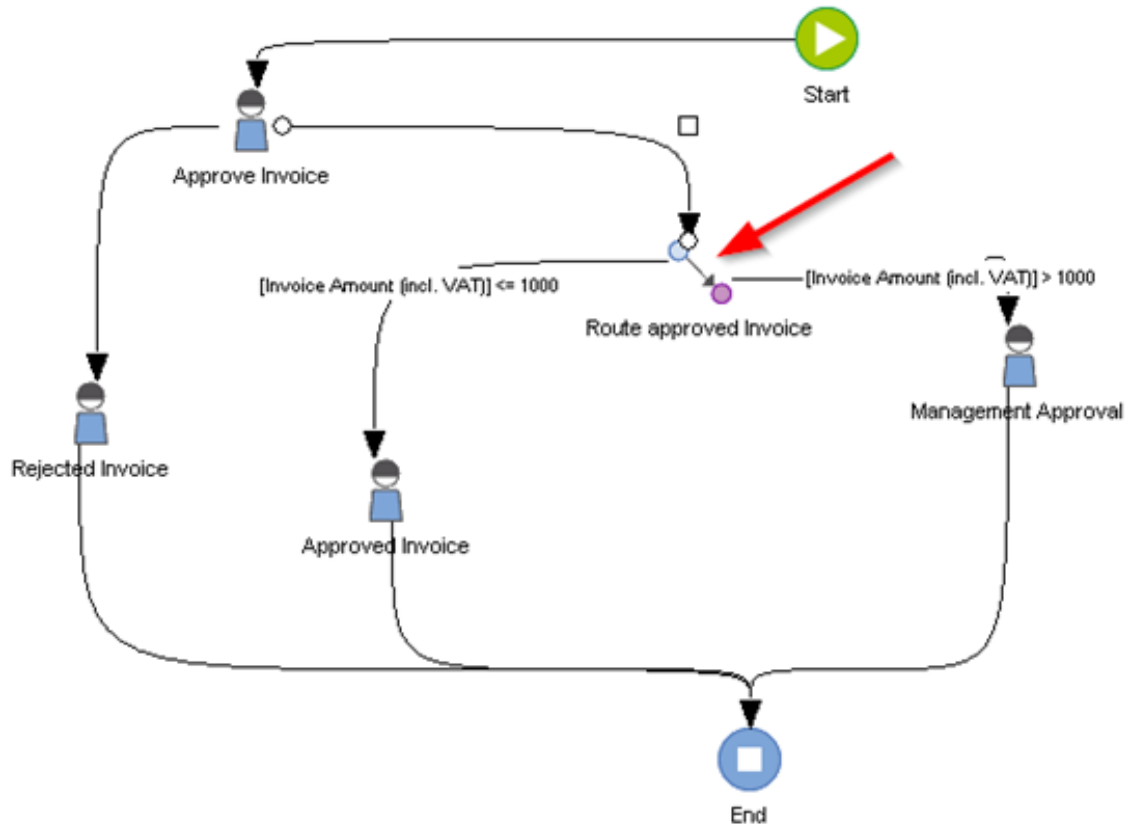


Based on the above workflow, the options under **Process/Finish Task** will be presented in the following format within the UI:

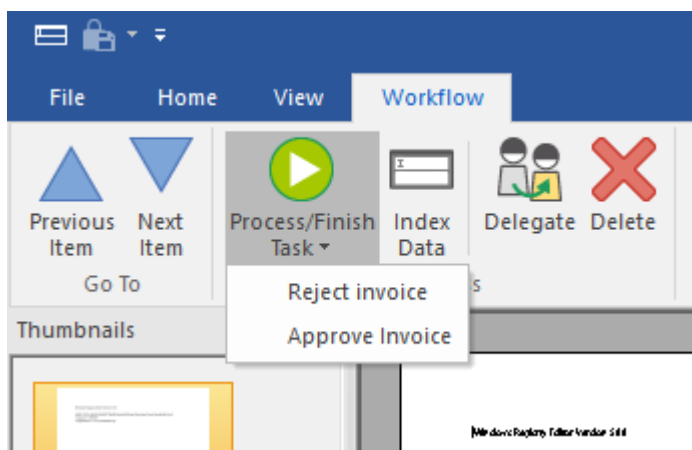


Since **Approve Invoice** and **Approve Invoice and send to Management** cannot be selected at the same time, there is no need to show both in the UI, as this could mislead the user.

A more appropriate pattern to the previous one could be presented as such:



Meanwhile, this improved version will be presented in the UI as:

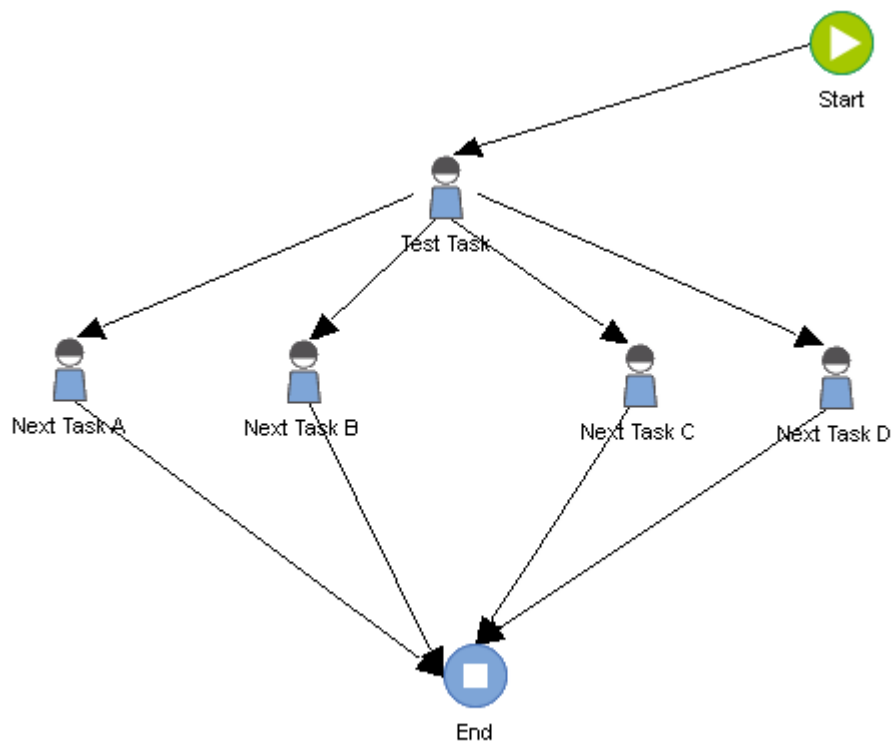


The workflows behave in the same way; however, the UI in the second example is much clearer to the user and redundant actions are not shown.

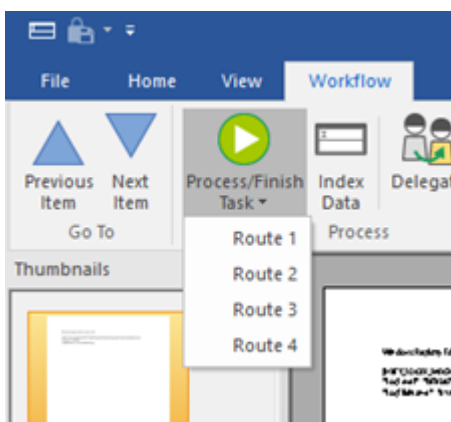
2.1.2 Example 2

In another example, there are four possible routes out of a single task, but only one route is valid (so there is no real decision for the user to take, it is actually determined by business rules). Without the pattern, the user sees four actions and receives an error message following the selection of one of the invalid routes.

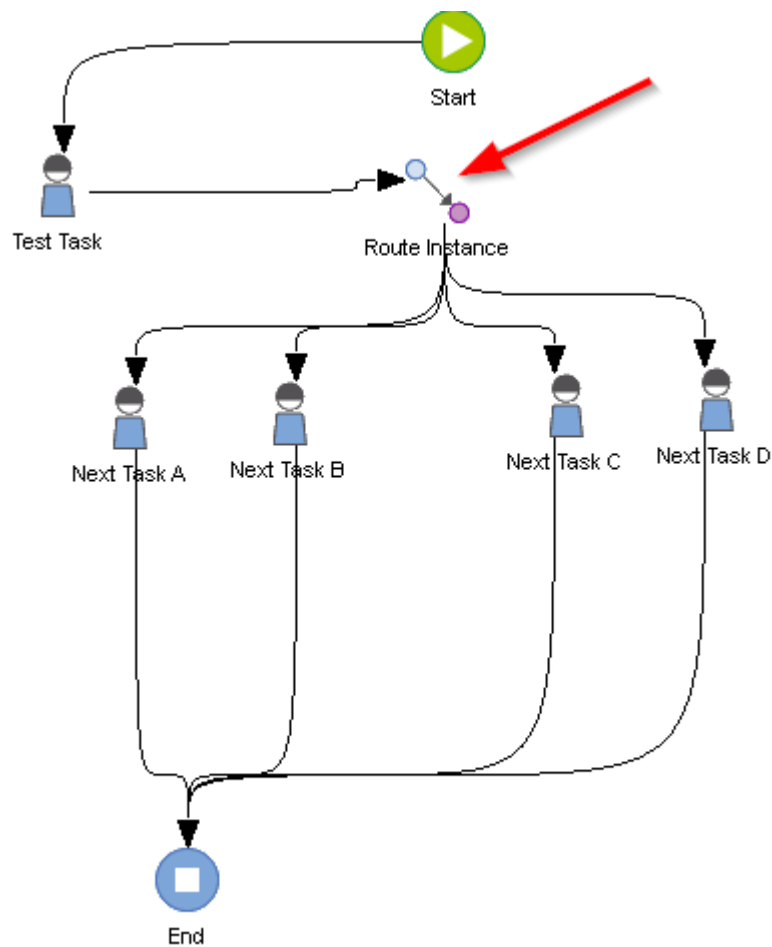
The below workflow is a representation of how this is commonly implemented:



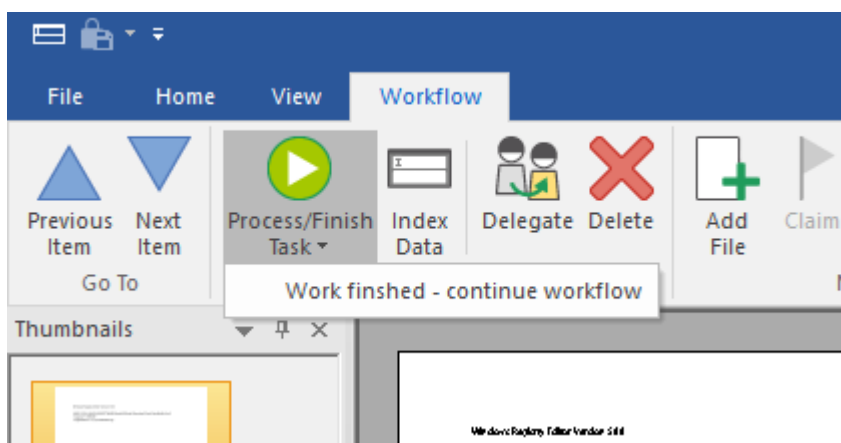
Based on the above workflow, the options under **Process/Finish Task** will be presented in the following format within the UI:



Nevertheless, this can be corrected by reducing to only one action:



This is how the improved version of the workflow will be represented in the UI:



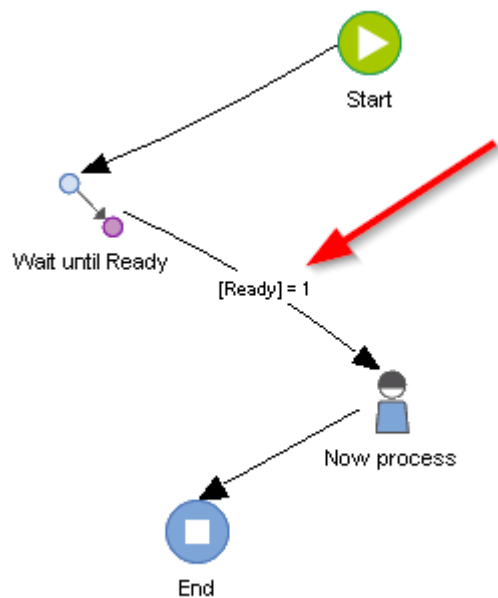
2.2 Pattern 2 - Wait for an index data change / flag

In this scenario, the purpose is to pause the workflow until a specific index data field is set. This can be a flag from an external system or a sub-workflow that indicates that this workflow can continue now. This can also be an index data value updated from outside the workflow by a user. The goal is to make the workflow pause until an index data field has a certain value.

2.2.1 Example

This can be achieved by using a routing task in combination with a condition. It is important that **"Treat as error if no valid transition is found"** is ticked **OFF**.

This acknowledges that it is a valid situation where the workflow cannot continue and will be blocked in this state as long as **Ready** is not set (to **1**). If this were to be ticked **ON**, it would be considered as a workflow error and reported as such.



The screenshot shows a 'Wait until Ready' task configuration window. The 'Task Definition' section includes:

- Name:** Wait until Ready
- Duration:** Fixed: 0 Days (selected), From index data: <None>
- Type:** Routing

An 'Automatic Configuration' sub-dialog is open, showing:

- Task type:** Routing
- Treat as error if no valid transition is found:** ☐ (highlighted with a red box)

At the bottom of the main window, it states: 'This workflow task can be referenced using task number '4494'. Unique ID: 45203865-D7EE-44A4-B4D0-87B169AECD2A'. There are 'OK' and 'Cancel' buttons at the bottom right.

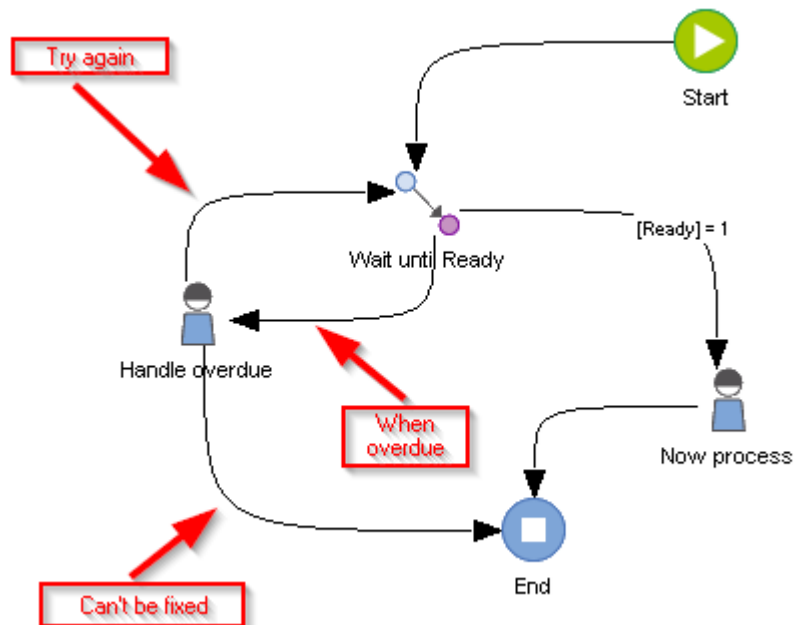
As a result, the workflow instance waits in this task quietly until the condition is met; then the workflow continues. Check if the condition is met on every run time of the background workflow task; this can be configurable, which by default is set to 15 minutes.

2.3 Pattern 3 – Handling automatic task overdues

Some automatic tasks may wait for some action to take place, e.g. a specific document to be saved or an index data field to be updated (see [Pattern 2](#)).

2.3.1 Example

However, it should be possible to react if this does not happen within a certain time frame. This is the correct pattern to handle automatic task overdue and escalations:



Define the time for when an automatic task will be overdue in the task settings:

Wait until Ready

Task Definition

Name:

Duration: ☒ Fixed:

☐ From index data:

Type:

This workflow task can be referenced using task number '4494'.
Unique ID: 45203865-D7EE-44A4-B4D0-87B169AEC2A

Define the route to be taken when overdue on the correct transition. For this it is necessary to tick the checkbox **"Use for overdue instances"** and the checkbox **"Make transition invisible"**. The **"Make transition invisible"** is required to not consider / ignore this transition as long as a task is **NOT** overdue.

Note: The **"Overdue for more than x days"** setting (grace period) is for manual task usage **ONLY** and will be ignored for automatic task overdue handling.

Transition Properties

Name: Go to overdue handling

Description:

Conditions:

Category

☐ Use this transition as ELSE path

Condition error text:

History text:

Additional settings:

☐ Request additional information from user

☒ Use transition for overdue instances

Overdue for more than 0 Days

☒ Make transition invisible

OK Cancel Validate Macro...

When configured like this, the workflow instance is routed to "Handle overdue" as soon as the 5 days period defined in the task settings has run out.

2.4 Pattern 4 – Handling of manual task overdues

If a manual task is assigned to a user, but the user is unable to process the item in time, it should be possible for the workflow to take an escalation route and inform management or other users about this situation and react on it.

2.4.1 Example

The setup of this pattern is almost equal to [Pattern 3](#) (overdue for automatic tasks), but there are some more options available when defining overdue processing for manual tasks.

Approve [X]

Task Definition | Assign To | Index Field Permissions | E-mail Notifications | Create Document

Name: Approve Invoice

Duration: ☒ Fixed: 5 Days ☐ From index data: <None>

User info: Please approve invoice

☐ Display as checklist

Options

- ☐ Do not send notification e-mails
- ☐ Do not send overdue e-mails
- ☐ Disable delegation
- ☐ Request re-authentication when finishing task
- ☐ Keep original version in document history
- ☐ Allow to delete unwanted documents
- ☐ Allow instance processing in Therefore™ Navigator

This workflow task can be referenced using task number '4164'.
Unique ID: F3C178AF-ABC7-47B4-BAA5-E73857B3CCA7

OK Cancel

It is possible and recommended to define a specific overdue message to be sent as email to the assigned user and maybe additional to the users management in order to inform the user about the issue.

The screenshot shows a software window titled "Approve" with a close button (X) in the top right corner. Below the title bar is a tabbed interface with five tabs: "Task Definition", "Assign To", "Index Field Permissions", "E-mail Notifications" (which is the active tab), and "Create Document".

Under the "E-mail Notifications" tab, there are three sections, each with a text label, a text area, and two buttons ("Edit..." and "Delete"):

- Notification e-mail:** this e-mail is sent when a new workflow instance is assigned to a user.
- Overdue e-mail:** this e-mail is sent when a workflow instance was not processed in time. (This entire section is highlighted with a red rectangular border in the image.)
- Delegate e-mail:** this e-mail is sent when a workflow instance is delegated.

At the bottom right of the window are "OK" and "Cancel" buttons.

E-mail Setup [X]

From: workflow@customer.net [OK]

To: [Currently assigned user is added automatically]

Cc: Manager@customer.com [Also inform manager]

Bcc:

Subject: Overdue on item [Invoice No]

Message: test

Attachments:

- ☒ Attach link to workflow instance

Name: WF [@ProcessName] [@InstanceNo].thex
- ☐ Attach link to main document

Name:
- ☐ Attach main document

Name:

[Cancel] [Macro...]

It is possible and recommended to define an additional grace period before the overdue route is finally taken and the instance is taken away from the user and routed to the escalation process. This grace period should give the user some more time to handle the issue after they have been informed about the overdue situation. Otherwise they may get informed about the item being overdue, but would not have a chance to react if the item was already moved forward into the escalation process. Hence, it is always recommended to use this grace period setting for manual task overdue handling. This setting will be ignored for automatic task overdue handling (nobody receives an email, not one could/would react on the overdue during a grace period – see [Pattern 3](#)).

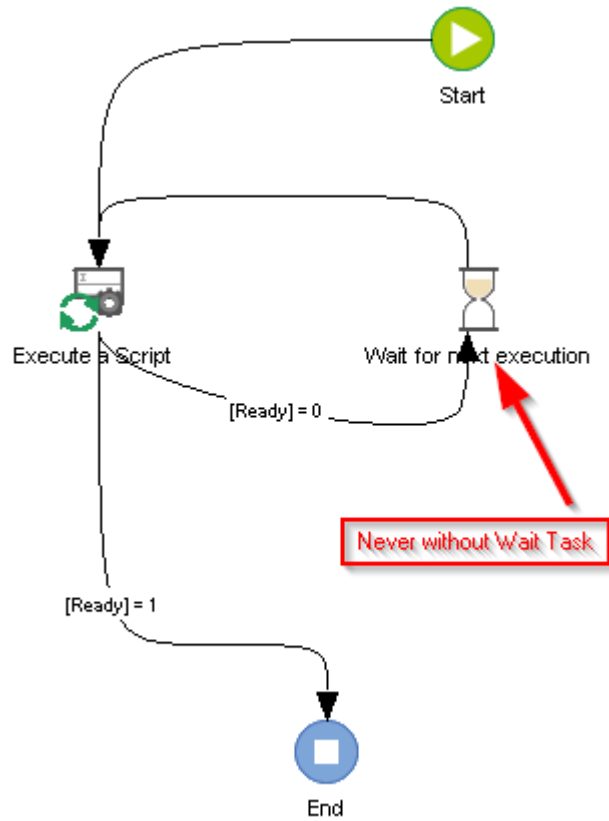
The screenshot shows the 'Transition Properties' dialog box. The 'Conditions' section is highlighted with a red box. It contains a list of conditions, with 'Overdue for more than' selected. The value '3' is entered in the adjacent input field, and 'Days' is selected in the dropdown menu. Below this, the 'Make transition invisible' checkbox is checked. Other fields in the dialog include 'Name', 'Description', 'Category', 'Condition error text', 'History text', and 'Additional settings' (with checkboxes for 'Request additional information from user' and 'Use transition for overdue instances'). Buttons for 'OK', 'Cancel', 'Validate', and 'Macro...' are on the right.

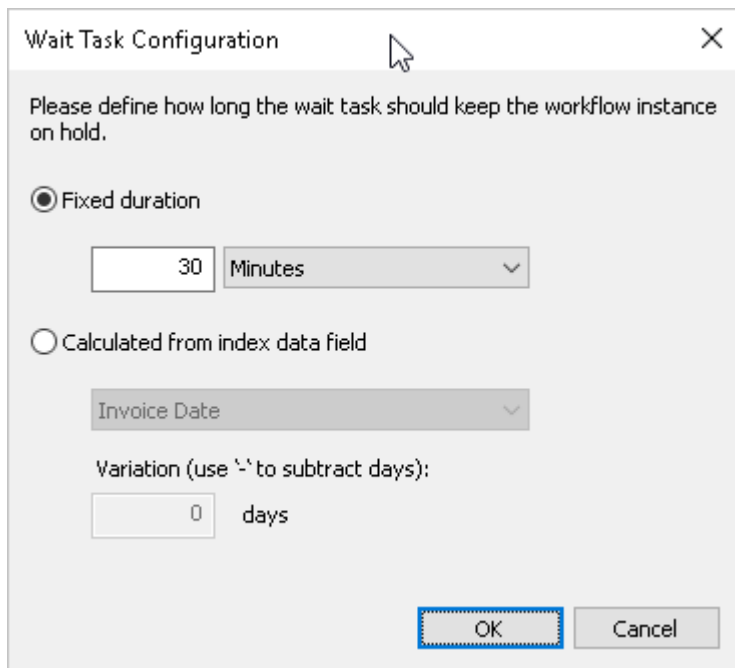
2.5 Pattern 5 – Execute script in a loop

This pattern is useful in case you need to run a script in a loop just as long as a certain condition can be met.

2.5.1 Example

This can be used for checking data in an external table or any other calculations. In the end the script needs to be executed in a loop until the final condition is satisfied.





The image shows a 'Wait Task Configuration' dialog box. It has a title bar with a close button (X). The main text says 'Please define how long the wait task should keep the workflow instance on hold.' There are two radio buttons: 'Fixed duration' (selected) and 'Calculated from index: data field'. Under 'Fixed duration', there is a text input field with '30' and a dropdown menu with 'Minutes'. Under 'Calculated from index: data field', there is a dropdown menu with 'Invoice Date' and a text input field with '0' followed by 'days'. At the bottom right, there are 'OK' and 'Cancel' buttons. The 'OK' button is highlighted with a blue dashed border.

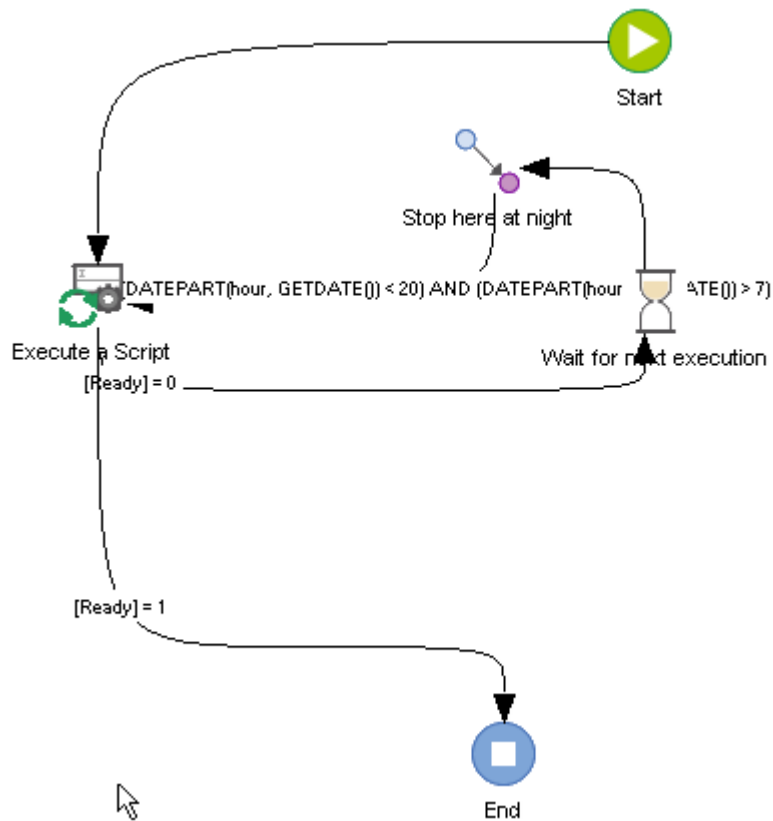
Important here is that a loop should always contain either a manual or a wait task to prevent countless executions in a very short timeframe.

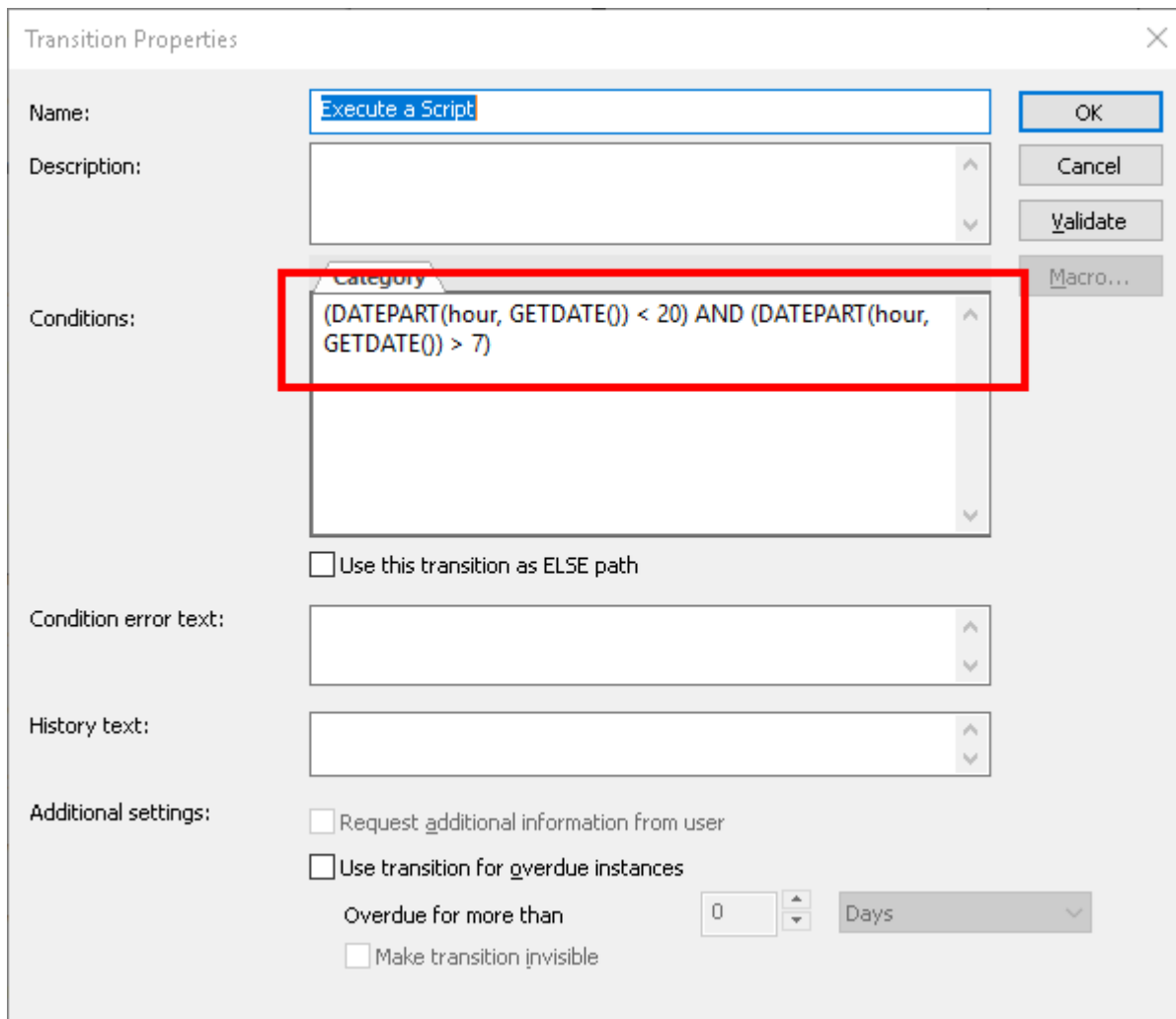
2.6 Pattern 6 – Execute script in a loop with advanced timing

See [Pattern 5](#), but additionally we want to pause the execution of the script out of office hours as we know the final condition can never be met out of office hours – no changes to the system are most probably made out of office hours which would satisfy the condition.

2.6.1 Example

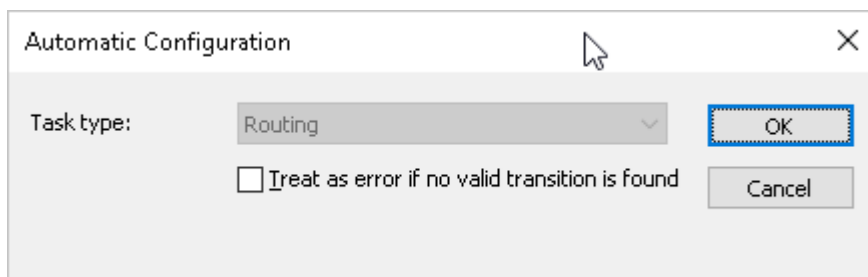
To prevent unnecessary executions of the script/task and keep the overall workflow healthy we want to suspend execution during night.





The image shows a 'Transition Properties' dialog box. The 'Name' field is 'Execute a Script'. The 'Conditions' field contains the SQL expression: `(DATEPART(hour, GETDATE()) < 20) AND (DATEPART(hour, GETDATE()) > 7)`. This field is highlighted with a red rectangle. Other fields include 'Description', 'Condition error text', 'History text', and 'Additional settings' with checkboxes for 'Use this transition as ELSE path', 'Request additional information from user', 'Use transition for overdue instances', and 'Make transition invisible'. The 'Overdue for more than' field is set to '0' days. Buttons on the right include 'OK', 'Cancel', 'Validate', and 'Macro...'.

Configuration of “**Stop here at night**” routing task – note the “**Treat as error if no valid transition is found**” is NOT ticked (see [Pattern 2](#)).



The image shows an 'Automatic Configuration' dialog box. The 'Task type' is set to 'Routing'. The checkbox 'Treat as error if no valid transition is found' is not checked. Buttons on the right include 'OK' and 'Cancel'.

The magic happens in the condition (works on SQL Server), which only evaluates true between 7:01 and 19:59. In all other cases the execution will be suspended.

Statement for office hours: `(DATEPART(hour, GETDATE()) < 20) AND (DATEPART(hour, GETDATE()) > 7)`

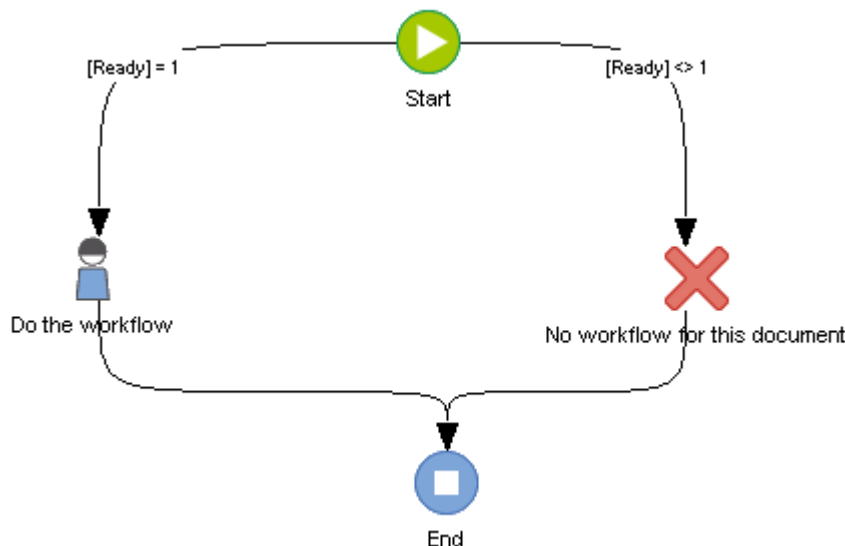
Statement for office hours and weekdays: (DATEPART(hour, GETDATE()) < 20) AND (DATEPART(hour, GETDATE()) > 7) AND (DATEPART(weekday, GETDATE()) > 1) AND (DATEPART(weekday, GETDATE()) < 7)

2.7 Pattern 7 – Do not start a workflow for certain documents

This pattern is for a case in which a user has a category and each new document to the category triggers a new workflow. In the end, the user only wants the workflow to be triggered for specific documents, for all others it should just be ignored.

2.7.1 Example

There is an easy pattern to solve this.



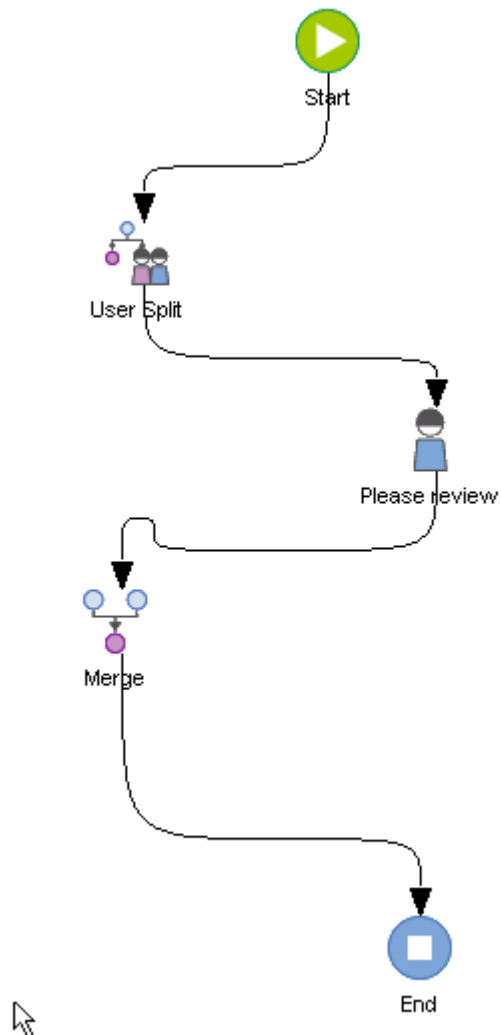
In this case only documents that satisfy the condition **Ready = 1** have a workflow initiated – for all other documents, the workflow is immediately deleted and removed from the system.

2.8 Pattern 8 – Several users implementing the same task in parallel

This pattern is for a case that one specific task needs to be done by multiple people in parallel, e.g. all need to approve the changes made to an order. The task is the same for all people that have to complete the task, and their number may be predefined or dynamic.

2.8.1 Example

For this pattern choose the **User Split Task**.



If the people that need to complete the task are known at design time, just define those people in the assign to field of the “**Please review**” task. Each of those persons will get a separate workflow token to be processed. Workflow will continue when each token arrived in **Merge** task.

Please review

Task Definition Assign To Index Field Permissions E-mail Notifications Create Document

☒ User/Groups

Display Name	Username
Forum Accounting	ADOS\FORUM.ACCOU...
Forum Customer	ADOS\FORUM.CUSTO...
Forum Legal	ADOS\FORUM.LEGAL

☐ User must choose from these users/groups

☐ Allow to select more than one user/group

☐ Expand groups on assignment

☐ User from index data field

Send To Mail

OK Cancel

If the users are not known at the design time, this can be achieved by assigning it from an index data field. This can either be a text field containing multiple usernames, each one separated by “,” or “;” (just like email recipients) or by referring to a multiple keyword field. Each of user will get a separate workflow token to be processed. The workflow will continue when each token arrives in the **Merge** task.

The screenshot shows a 'Please review' dialog box with a close button (X) in the top right corner. The 'Assign To' tab is selected, and a mouse cursor is pointing at it. The dialog contains the following elements:

- Tab bar: Task Definition, Assign To (selected), Index Field Permissions, E-mail Notifications, Create Document.
- Radio button: ☐ User/Groups
- Table:

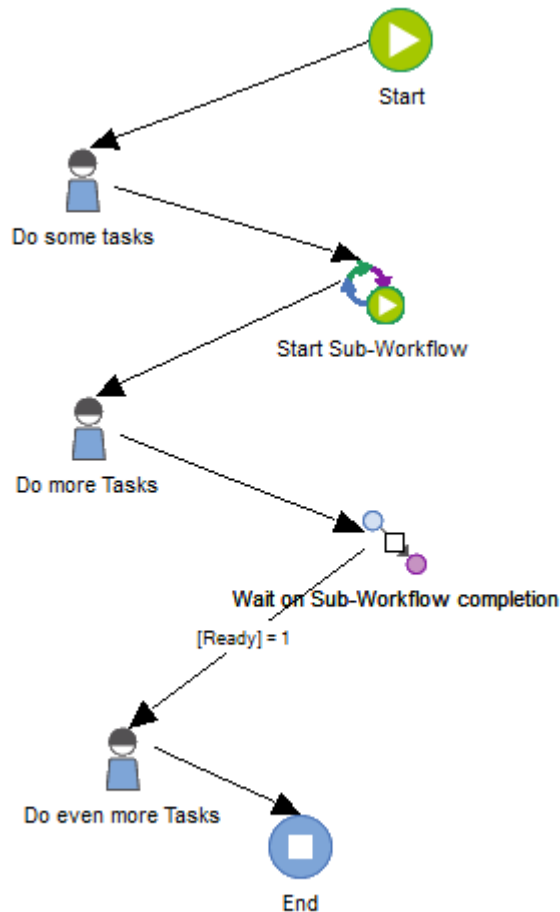
Display Name	Username
- Buttons: ... (next to the table)
- Checkboxes:
 - ☐ User must choose from these users/groups
 - ☐ Allow to select more than one user/group
 - ☐ Expand groups on assignment
- Radio button: ☒ User from index data field
- Dropdown menu: Send To Mail (with a downward arrow)
- Buttons: OK, Cancel

2.9 Pattern 9 – Synchronizing Sub-workflows

This pattern is for synchronizing sub-workflows with the main workflow of a document. This is helpful when the user wants some tasks to be completed in a sub-workflow, but also want at a certain point make sure the main workflow waits until the sub-workflow has completed its work.

2.9.1 Example

The main workflow looks like:



Note: “Wait on Sub-Workflow completion” makes use of [Pattern 2](#) and the condition for this is to wait until Ready = 1.

Automatic Configuration

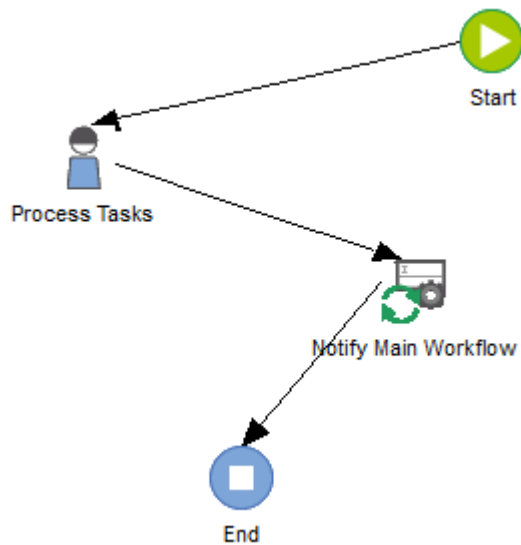
Task type:
Routing

☐ Treat as error if no valid transition is found

OK

Cancel

The sub-workflow looks like:



The Task “**Notify Main Workflow**” sets the Ready flag to 1.

Category Field	Assignment
Invoice No	...
Invoice Date	...
Invoice Amount (net)	...
Invoice Amount (incl. VAT)	...
Customer Name	...
Customer Address	...
Ready	1

This pattern makes the main workflow pause in task “**Wait on sub-workflow completion**” until the flag is set to 1 by the sub-workflow; hence, the flow of execution can be synchronized between main and sub-workflow.

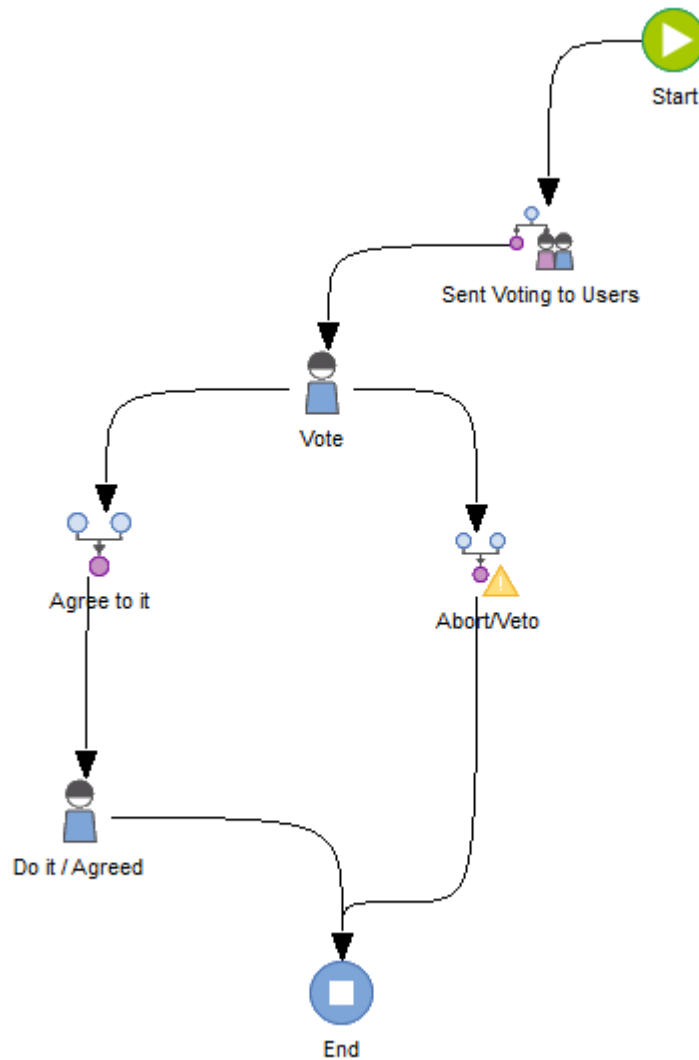
2.10 Pattern 10 - Voting

This pattern is for implementing a type of voting into the Therefore™ Workflow (Therefore™ 2018 and newer). Voting can be used to determine if an action should continue or a route should be taken by a minimum amount of users taking on a specific decision. E.g. a User Split could create tokens for a flexible amount of users to take a decision, and if at least 2 users voted for 'Yes' or 'No' the specific route is taken – the other tokens are deleted because no more votes are required.

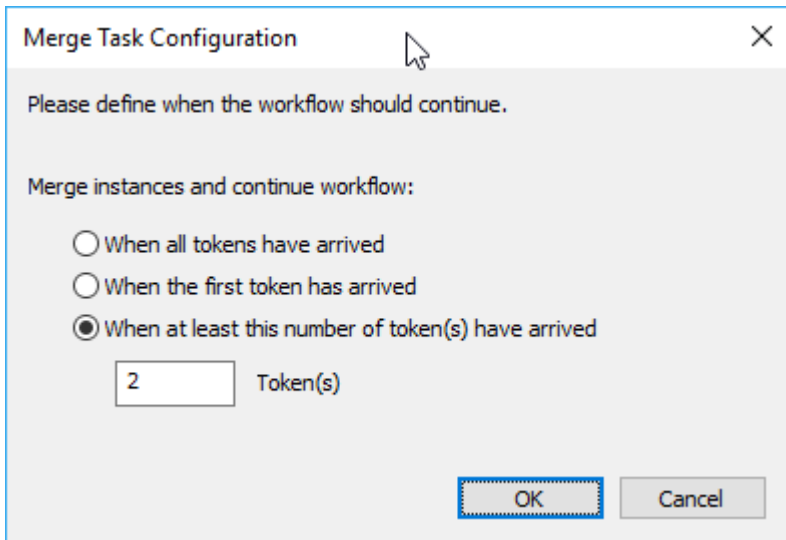
In the sample workflow, we need to agree if a certain action should be done. We would need at least two votes to accept that. Anyone that would be against it can abort the action through an exception merge. So if we ask 5 people if they would agree with a certain action, the workflow would continue as soon as two have agreed. The other votes are ignored/removed.

2.10.1 Example 1

An example of such a workflow can be built in the following manner:



The details for the merge task, “Agree to it” are defined as:



Merge Task Configuration

Please define when the workflow should continue.

Merge instances and continue workflow:

- ☐ When all tokens have arrived
- ☐ When the first token has arrived
- ☒ When at least this number of token(s) have arrived

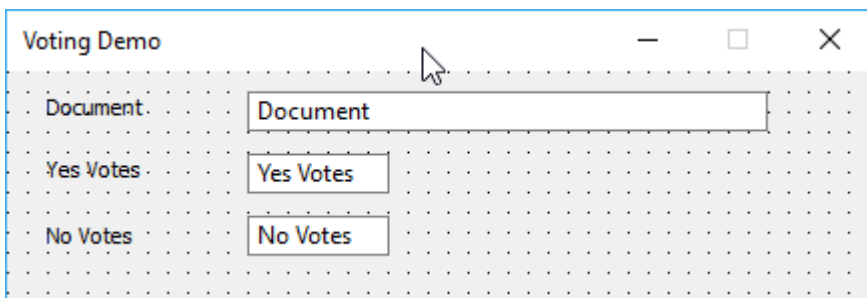
Token(s)

OK Cancel

2.10.2 Example 2 (Advanced)

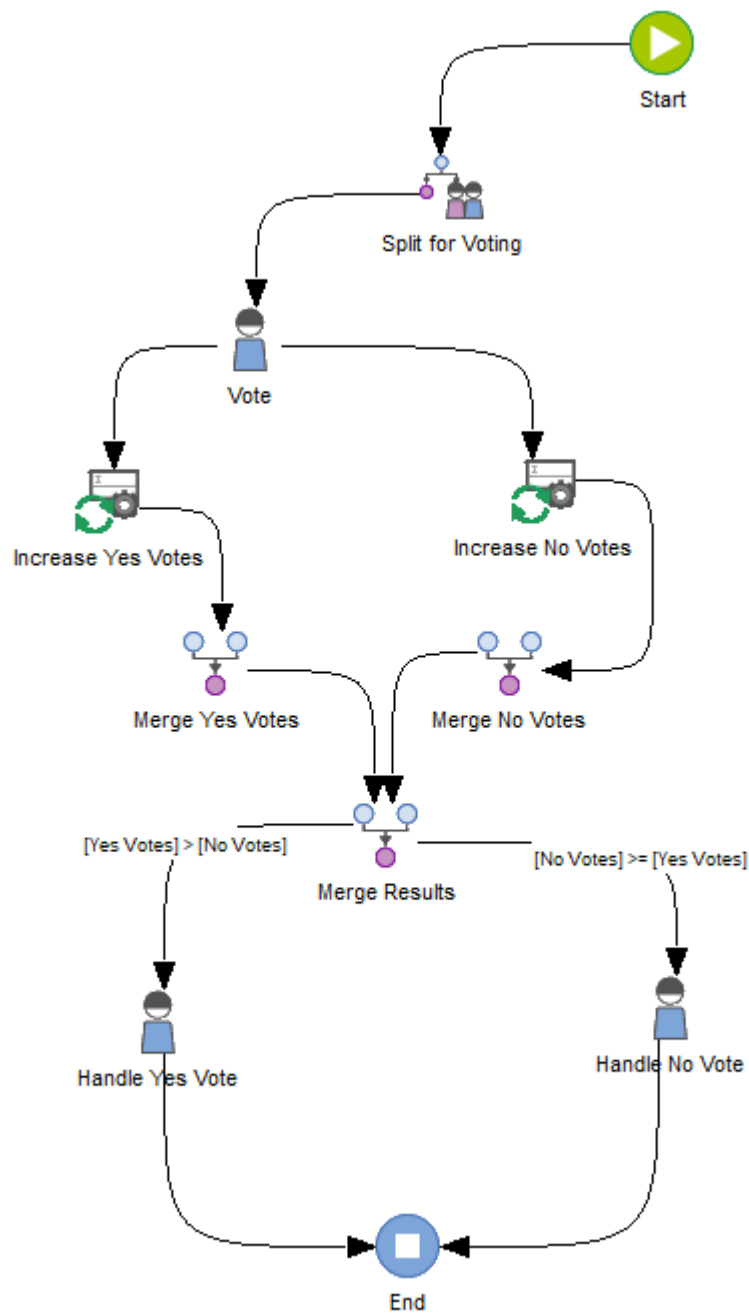
This pattern is for implementing voting into the Therefore™ Workflow (Therefore™ 2018 and newer). Voting can be used to determine whether an action should continue or a route should be taken by an minimum amount of users taking a specific decision. E.g. a User Split could create tokens for a flexible number of users to take a decision, and if at least 2 users voted for 'Yes' or 'No', the specific route is taken – the remaining tokens are deleted because no further votes are required.

This is precise voting, so tracking the number of 'Yes' and 'No' votes is required in order to enable the selection of the correct path once the results have been merged. The category needs to have an index data field for 'Yes' and 'No' vote counts.

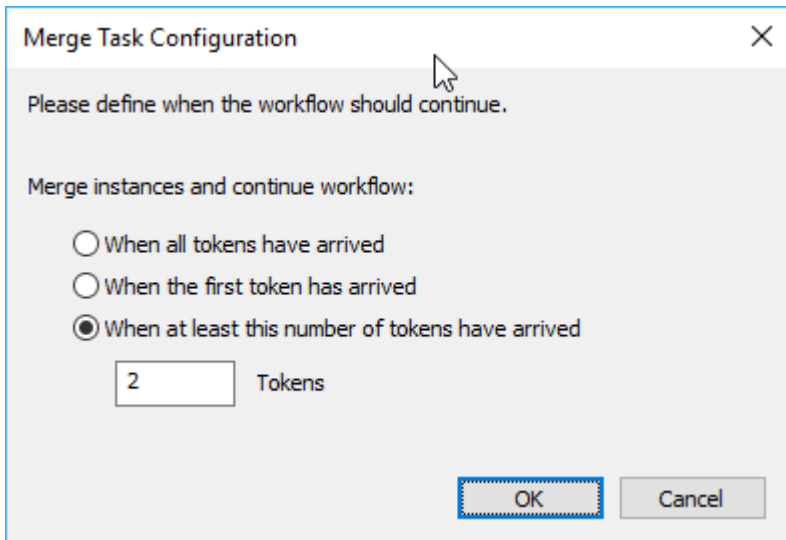


Voting Demo

Document	Document
Yes Votes	Yes Votes
No Votes	No Votes



The settings for the Merge 'Yes' and Merge 'No' Votes Task are:



Merge Task Configuration

Please define when the workflow should continue.

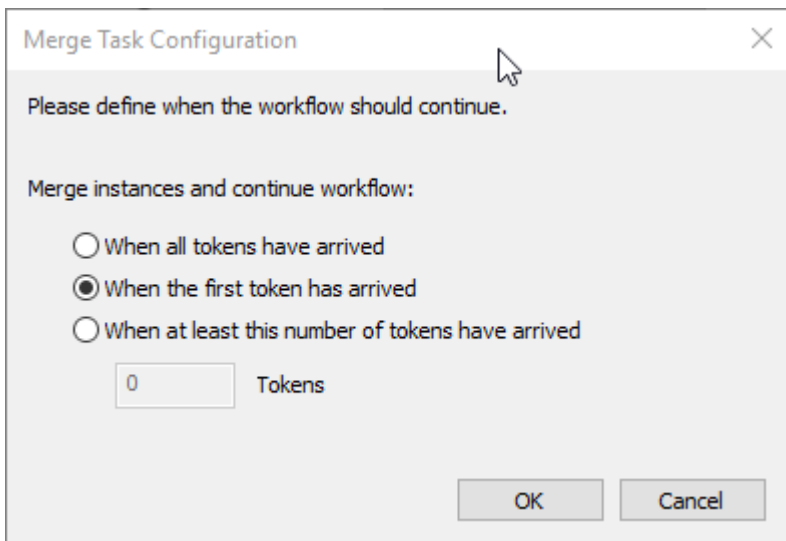
Merge instances and continue workflow:

- ☐ When all tokens have arrived
- ☐ When the first token has arrived
- ☒ When at least this number of tokens have arrived

Tokens

OK Cancel

As soon as two people voted 'Yes' or 'No', the workflow advances to the Merge Result Task. The setting for this Task is:



Merge Task Configuration

Please define when the workflow should continue.

Merge instances and continue workflow:

- ☐ When all tokens have arrived
- ☒ When the first token has arrived
- ☐ When at least this number of tokens have arrived

Tokens

OK Cancel

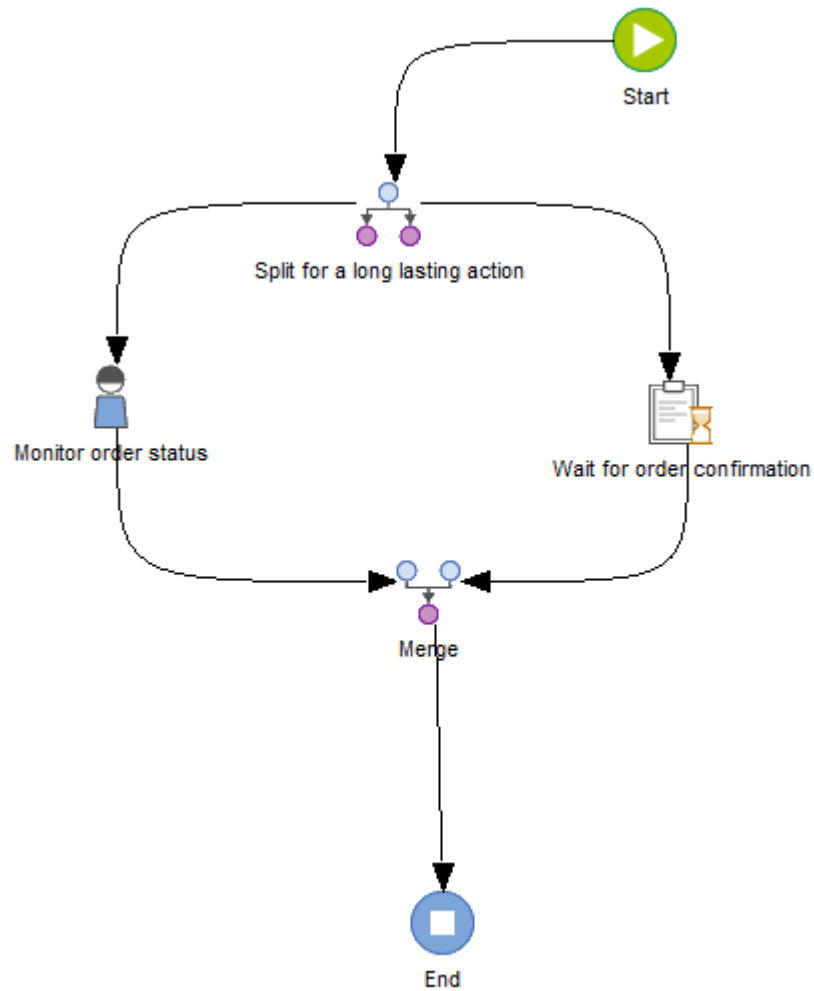
2.11 Pattern 11 - Abort a long lasting action

This pattern is for implementing a way to abort a long lasting automatic action in Therefore™ Workflow (Therefore™ 2018 and newer). E.g. the use case is a long lasting wait task that could require cancellation on certain occasions. For now it was required to reroute the workflow task by a process owner to archive this, but by using the pattern below, a regular user could abort this action though regular workflow processing.

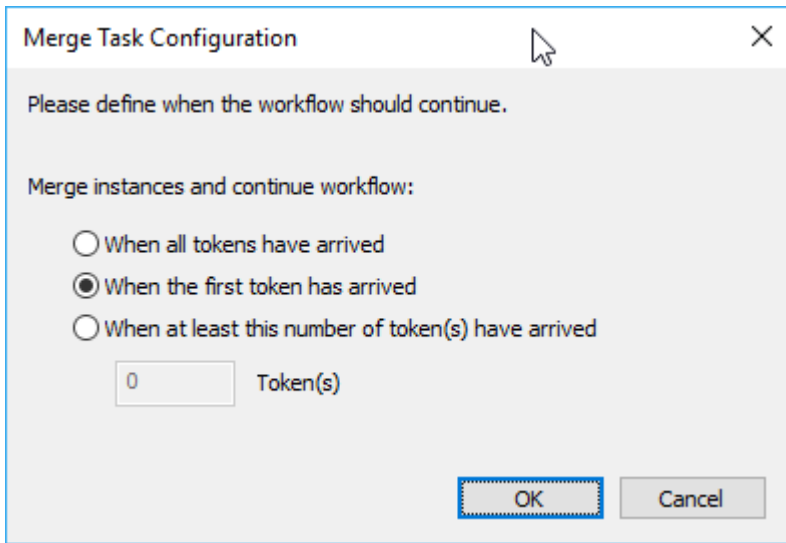
In the following sample, we have an order processing flow. We want to wait for the order confirmation by using a wait document task. But in case the order is canceled, we also want to be able to abort the wait operation and continue with the workflow.

2.11.1 Example

An example of such a workflow can be constructed as such:



The details for the merge task, "Merge" are defined as:



Merge Task Configuration

Please define when the workflow should continue.

Merge instances and continue workflow:

☐ When all tokens have arrived

☒ When the first token has arrived

☐ When at least this number of token(s) have arrived

0 Token(s)

OK Cancel

When a token either arrives from “Wait for order confirmation” (because the confirmation was saved to Therefore™) or the order was canceled through the manual task, the workflow is continued and the other token is cancelled. This is also defined as OR-Merge.